

CHAPITRE 5 : JAVA SCRIPT

INTRODUCTION

JavaScript, créé en 1995 par Brendan Eich (pour la Netscape Communication Corporation. Il est souvent décrit comme étant un langage de script dynamique. C'est un langage de programmation de scripts orienté objet.

Les scripts Javascript :

- Sont de simples fichiers "texte" (extension conseillée .js) à créer avec un éditeur de texte.
- Sont intégrés au sein des pages web.
- Sont exécutés côté client par le navigateur web.

JavaScript sur le Web est capable :

- Valider des saisies
- Créer des effets d'enroulement
- Créer des menus déroulants ou contextuels
- Gérer le glisser et déposer
- Faire défiler des pages Web à l'infini
- Compléter automatiquement des champs
- Créer des listes pouvant être triées

I. POURQUOI JAVA SCRIPT ?

JavaScript est devenu le standard pour créer des interfaces utilisateur dynamiques pour le Web. Pratiquement chaque fois que vous visitez une page Web qui contient des animations, des données évolutives, un bouton dont l'aspect change lorsque vous le survolez, ou encore un menu déroulant, vous pouvez être certain que JavaScript est entré en action. Du fait de sa puissance et de sa capacité à s'exécuter dans virtuellement n'importe quel navigateur Web, JavaScript est devenu l'outil le plus populaire et le plus nécessaire que tout développeur Web moderne doit utiliser.

Java Script n'est pas JAVA :

Javascript

Code intégré dans la page Html
Code interprété par le browser au moment de l'exécution
Codes de programmation simples mais pour des applications limitées
Permet d'accéder aux objets du navigateur
Confidentialité des codes nulle (code source visible)

Java

Module (applet) distinct de la page Html
Code source compilé avant son exécution
Langage de programmation beaucoup plus complexe mais plus performant
N'accède pas aux objets du navigateur
Sécurité (code source compilé)

II. Éditeurs de texte pour le développement

Nom	Adresse Web
Coda	http://panic.com/coda
Aptana	www.aptana.com
Komodo Edit	www.activestate.com/komodo-edit/download
Dreamweaver	http://adobe.com/products/dreamweaver.htm
Eclipse	www.eclipse.org
Notepad ++	http://notepad-plus-plus.org
TextMate	http://macromates.com
BBEdit	www.barebones.com/products/bbedit
EMacs	www.gnu.org/software/emacs
TextPad	www.textpad.com
Vim	www.vim.org
Netbeans	https://netbeans.org

III. Syntaxe

III.1 La balise <SCRIPT>

De ce qui précède, vous savez déjà que votre script vient s'ajouter à votre page Web.

Le langage Html utilise des tags ou balises pour "dire" au browser d'afficher une portion de texte en gras, en italique, etc.

Dans la logique du langage Html, il faut donc signaler au browser par une balise, que ce qui suit est un script et que c'est du Javascript (et non du VBScript). C'est la balise

<SCRIPT LANGUAGE="Javascript">.

De même, il faudra informer le browser de la fin du script. C'est la balise </SCRIPT>

III.2 les commentaires

Il vous sera peut-être utile d'inclure des commentaires personnels dans vos codes Javascript. C'est même vivement recommandé comme pour tous les langages de programmation.

Javascript utilise les conventions utilisées en C et C++ soit // commentaire

Tout ce qui est écrit entre le // et la fin de la ligne sera ignoré.

Il sera aussi possible d'inclure des commentaires sur plusieurs lignes avec le code

/* commentaire sur plusieurs lignes */

Ne confondez pas les commentaires Javascript et les commentaires Html

(pour rappel <!-- ...-->)

III.3 Masquer le script pour les anciens browsers

Les browsers qui ne comprennent pas le Javascript (et il y en a encore) ignorent la balise <script> et vont essayer d'afficher le code du script sans pouvoir l'exécuter. Pour éviter l'affichage peu esthétique de ses inscriptions cabalistiques, on utilisera les balises de commentaire du langage Html <!-- ... -->

Votre premier Javascript ressemblera à ceci :

```
<SCRIPT LANGUAGE="javascript">
<!-- Masquer le script pour les anciens browsers
...
programme Javascript
...
// Cesser de masquer le script -->
</SCRIPT>
```

La syntaxe du Javascript n'est pas compliquée. De manière générale, les instructions doivent être séparées par un point-virgule que l'on place à la fin de chaque instruction :

```
<script>
  instruction_1;
  instruction_2;
  instruction_3;
</script>
```

La syntaxe d'une fonction se compose de deux choses : son nom, suivi d'un couple de parenthèses (une ouvrante et une fermante). Entre les parenthèses se trouvent les arguments, que l'on appelle aussi paramètres.

```
<script>
  myFunction();
</script>
```

Par exemple :

```
<script>
  alert('Bonjour');
// la fonction affiche une boîte de dialogue avec "Bonjour"
</script>
```

III.4 Comment intégrer du JavaScript à du HTML ?

Le principe est simple. Il suffit de respecter les deux principes suivants :

- ✓ N'importe où.
- ✓ Mais là où il le faut.

Le browser traite votre page Html de haut en bas (y compris vos ajoutes en Javascript). Par conséquent, toute instruction ne pourra être exécutée que si le browser possède à ce moment précis tous les éléments nécessaires à son exécution. Ceux-ci doivent donc être déclarés avant ou au plus tard lors de l'instruction.

- Directement dans la page

Balise script

`<script> </script>`

cette balise peu être utilisée pour n'importe quel langage de scripting (comme du VBScript par exemple)

hormis avec HTML5, il est donc recommandé de préciser l'attribut `type="text/javascript"`

Exemple : Hello World !

```

1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Hello World!</title>
5    </head>
6
7    <body>
8
9      <script>
10
11        alert('Hello world!');
12
13      </script>
14
15    </body>
16  </html>

```

OU

```

1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
2  <html>
3    <head>
4      <title>Javascript XHTML</title>
5    </head>
6
7    <body>
8
9      <script type="text/javascript">
10
11        <!--
12        if (2 > 0){
13          alert('2 est bien supérieur à 0');
14        }
15        <!-->
16      </script>
17
18    </body>
19  </html>

```

ou

```

1  !DOCTYPE html>
2  <html>
3  <meta http-equiv="Content-Type"
4  content="text/html; charset=UTF-8" />
5  <head>
6  <title>Bonjour, HTML!</title>
7  </head>
8  <body>
9  <h1>Comptons jusqu'à 10 avec JavaScript!</h1>
10 <p id="theCount"></p>
11 <script>
12   var count = 0;
13   while (count < 10)
14   {
15     count++;
16     document.getElementById("theCount").innerHTML
17     += count + "<br>";
18   }
19 </script>
20 </body>
21 </html>

```

➤ Fichier séparé

Syntaxe

Attribut src de la balise source.

Le fichier .js utilisé ne nécessite aucune syntaxe particulière.

Exemple

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Hello World!</title>
5  </head>
6
7  <body>
8
9    <script src="helloworld.js"> </script>
10
11  </body>
12 </html>

```

➤ Placer directement le code dans un attribut d'événement HTML.

HTML possède plusieurs attributs spéciaux qui sont conçus pour déclencher du code JavaScript lorsque quelque chose de particulier se passe dans la fenêtre du navigateur, ou lorsque

l'utilisateur effectue une certaine action. Voici un exemple qui montre un bouton HTML possédant un attribut d'événement capable de répondre à des clics de souris :

```
<button id="bigButton"
onclick="alert(Bonjour, le Monde
!');">Cliquez ici</
button>
```

Lorsque l'utilisateur clique sur le bouton créé par cet élément HTML, une fenêtre pop-up affichant le message spécifié apparaît.

HTML possède environ 70 attributs d'événements différents. Le Tableau ci-dessous liste ceux qui sont le plus couramment employés.

Attribut	Description
onload	Exécute le script une fois le chargement des pages terminé.
onfocus	Exécute le script lorsque l'élément concerné a le focus (par exemple lorsqu'un champ de texte est actif).
onblur	Exécute le script lorsque l'élément concerné perd le focus (par exemple lorsque l'utilisateur clique sur un nouveau champ de texte dans un formulaire).
onchange	Exécute le script lorsque la valeur d'un élément change.

III.5 Où positionner la balise script ?

Très souvent, il est conseillé de placer vos balises scripts dans l'entête du code HTML (head).

Ce n'est pas forcément le meilleur choix.

Une page web est analysée de manière séquentielle. Donc si un javascript volumineux est déclaré en début de page, celui-ci va être chargé avant les éléments à afficher, ce qui risque de ralentir la navigation. Il est donc recommandé de placer vos balises script en fin de page.

Les navigateurs modernes ont tendance à, de toute manière, charger le javascript en dernier, quel que soit sa position dans la page ... mais vous n'êtes pas sans savoir que nous sommes dans l'obligation d'encore et toujours prendre en compte les navigateurs obsolètes.

- Insertion de code javascript dans une page web

```
<HTML>
  <HEAD>
    <TITLE>Exemple 1</TITLE>
    <SCRIPT TYPE="text/javascript">
      // Du code Javascript
      alert("Hello world !"); // affiche une boîte de dialogue modale
    </SCRIPT>
  </HEAD>
  <BODY>
  </BODY>
</HTML>
```

- Écrire dans une page web

L'élément `NOSCRIPT` permet de fournir un contenu de remplacement pour les navigateurs qui ne peuvent exécuter un script

```
<HTML>
  <HEAD>
    <TITLE>Exemple 2</TITLE>
    <SCRIPT TYPE="text/javascript">
      <!--
      document.write("<P>Votre navigateur accepte le Javascript.</P>");
      //-->
    </SCRIPT>
    <NOSCRIPT><P>Votre navigateur n'accepte pas le Javascript.</P></NOSCRIPT>
  </HEAD>
  <BODY>
  </BODY>
</HTML>
```

- Intégrer un script Javascript dans une page

Le programmeur préférera souvent rassembler son code javascript dans des fichiers de scripts qu'il intégrera alors dans les pages web de son site de la manière suivante :

```
<HTML>
  <HEAD>
    <TITLE>Exemple 3</TITLE>
    <SCRIPT TYPE="text/javascript" SRC="monscript.js"></SCRIPT>
  </HEAD>
  <BODY>
  </BODY>
</HTML>
```


IV. LES VARIABLES

Les variables sont les lieux de mémorisation des chiffres et des caractères. En utilisant l'assignement on attribue une valeur à une variable. La structure générale de l'assignement est `target = source`.

Pour assigner une chaîne de caractères, on la place entre guillemets doubles ("...") ou simples ('...').

Exemple :

a = 3

b = "hello world"

c = 'hello from Mars'

a = b

a = 3*4*3+2

Il existe des notations abrégées pour certains types d'assignements. Par exemple :

Notation abrégée	Signification
<code>x += y</code>	<code>x = x + y</code>
<code>x -= y</code>	<code>x = x - y</code>
<code>x++</code>	<code>x = x + 1</code>
<code>x--</code>	<code>x = x - 1</code>

En Javascript, il n'y a pas besoin de déclarer le type de variables que l'on utilise, contrairement à des langages évolués tels que le langage C ou Java pour lesquels il faut préciser s'il s'agit d'entier (int), de nombre à virgule flottante (float), de caractères (char), etc.

En fait, Javascript n'autorise la manipulation que de 4 types de données :

- Des nombres : entiers ou à virgules
- Des chaînes de caractères (string) : une suite de caractères
- Des booléens : des variables à deux états permettant de vérifier une condition :
 - ✓ False : lors d'un résultat faux
 - ✓ true: si le résultat est vrai
- Des variables de type null : un mot caractéristique pour indiquer qu'il n'y a pas de données.

IV.1 Opérations

IV.1.1 Les opérateurs arithmétiques

Les 4 opérations de base sont disponibles en mode texte :

❖ +
❖ -
❖ *
❖ /

Exemples :

somme = somme + 1

b = c - d

somme = somme ^ 5

*resultat = (3 + 5) * (23 / 4) - 3*

IV.1.2 Les opérateurs de comparaison

Les opérateurs de comparaison servent à exprimer des conditions. Attention à ne pas confondre l'opérateur de comparaison == avec le signe = d'assignation.

égal à	==
différent de	!=
supérieur à	>
supérieur ou égal à	>=
inférieur ou égal à	<
inférieur ou égal à	<=

IV.1.3 Les opérateurs logiques

ET logique	&&
OU logique	
NON logique	!

Exemple :

```
2  <script>
3  .....
4  var number1 = 3,
5  number2 = 2, result;
6  result = number1 *
7  number2;
8  alert(result); //
9  Affiche : « 6 »
10 </script>
```

```
2  <script>
3  var number = 3;
4  number = number + 5;
5  alert(number); //
6  Affiche : « 8 »
7  </script>
8  |
```

```
2  <script>
3  .....
4  var number = 3;
5  number += 5;
6  alert(number); //
7  Affiche : « 8 »
8  </script>
```

IV.1.4 La concaténation

Une concaténation consiste à ajouter une chaîne de caractères à la fin d'une autre, comme dans cet exemple :

```
2  <script>
3  var hi = 'Bonjour ', name = 'toi', result;
4  result = hi + name;
5  alert(result);
6  // Affiche : « Bonjour toi »
7  </script>
8
```

```
2  <script>
3  var text = 'Bonjour ';
4  text += 'toi';
5  alert(text);
6  // Affiche « Bonjour toi ».
7  </script>
```

On peut convertir un nombre en chaîne de caractères avec l'astuce suivante :

```
2  <script>
3  var text, number1 = 4, number2 = 2;
4  text = number1 + '' + number2; /* on ajoute une chaîne de caractères vide
5  entre les deux nombres pour permettre une concaténation sans calcul */
6  alert(text); // Affiche : « 42 »
7  </script>
```

Exercice 1 : Manipulation de variables

- Pour commencer, ouvrir votre éditeur de texte ou votre IDE, et créer un premier fichier nommé variables.html
- Créer la structure de votre HTML5.
- Dans une balise script, créer une variable et affectez lui "valeur 1" « maVariable» afficher son type
- Demander à l'utilisateur de rentrer un nombre
- Affichez "vous avez entré : " suivi du nombre entré

Indice :

Les méthodes suivantes vous seront utiles :

- alert('texte') : pour afficher une boîte de dialogue contenant le texte en argument
- parseInt('texte') : pour extraire une valeur numérique de la chaîne de caractère 'texte'
- Prompt('texte') : pour afficher une boîte de dialogue demandant à un utilisateur d'entrer un texte. Cette fonction prend pour argument le message à afficher (String) et renvoi une chaîne de caractère.

Exercice 2 : Manipulation de variables

Créez une page HTML avec une balise script. Dans cette balise script, déclarez deux variables, en leur affectant deux valeurs de votre choix, puis créer une ou plusieurs alert() afin d'afficher le résultat et le type d'une ou plusieurs opération(s) entre ces valeurs. Modifiez les valeurs, et leur type, ainsi que les opérateurs utilisés, autant de fois que nécessaire afin de bien comprendre le résultat des différentes conversions implicites de types, ainsi que l'utilité de chaque opérateur.

Afin d'en profiter pour vérifier vos autres connaissances, évitez autant que possible de consulter l'indice.

Indice :

Exemple de document HTML5 à réaliser, permettant de voir le résultat de "l'addition" entre un entier et une chaîne de caractère :

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Test JavaScript</title>
5  </head>
6
7  <body>
8    <script>
9      var v1 = 1;
10     var v2 = '2';
11     var rslt = v1 + v2 ;
12     alert(rslt);
13     alert(typeof rslt)
14   </script>
15 </body>
16 </html>

```

IV.2 Les choix (sélecteurs)

Choix avec une alternative

Formulation générale :

```
if (<condition>) {
```

```
<action>;
```

```
<action>;
```

```
...
```

```
}
```

Exemple :

```
if ( j == 5 ) {
```

```
Somme = Somme +1;
```

```
}
```

choix avec deux alternatives

Formulation générale :

```
if (<condition>) {
```

```
<action>;
```

```
<action>;
```

```
...
```

```
} else {
```

```
<action>
```

```
<action>
```

```
...
```

```
}
```

Exemple :

```
if ( j == 5 ) {
```

```
Somme = Somme + 1 ;
```

```
} else {
```

```
Somme = Somme - 1 ;
```

```
}
```

Choix avec alternatives reliées

Formulation générale :

```
if ( <cond1> <opérateur> <cond2>
```

```
) {
```

```
<action>
```

```
<action>
```

```
...
```

```
}
```

Exemples :

```
if ( NP < 4000 && NP >= 3900 ) {
```

```
Canton = "Valais";
```

```
}
```

```
if ( !(x < 5 && y > 7) ) {
```

```
resultat = "correct";
```

```
}
```

Choix avec conditions imbriquées

Formulation :

```
if (<condition1>) {
  <action>
} else {
  if (<condition2>) {
    <action>
  }
  <action>
  ...
} else {
  <action>
  <action>
  ...
}
```

Exemple :

```
if ( reponse ==1 ) {
  cadeau = "cigarettes";
} else {
  if ( reponse==3 ) {
    cadeau = "fleurs" ;
  } else {
    cadeau = "chaussettes" ;
  }
}
```

La fonction confirm() permet une interaction de l'utilisateur à l'exécution du code (true si OK, false si Annuler) :

```
1 <script>
2 if (confirm('Voulez-vous exécuter le code Javascript de cette page ?')) {
3   alert('Le code a bien été exécuté !');
4 }
5 </script>
```

La structure else permet de simplifier l'alternative :

```
1 <script>
2 if (confirm('Pour accéder à ce site vous devez être une fille, cliquez sur un
3 "OK" si c\'est le cas.')) {
4   alert('Vous allez être redirigé vers le site.');
```

On peut ajouter des conditions intermédiaires avec la structure else if :

```
1 <script>
2 var floor = parseInt(prompt("Entrez l'étage où l'ascenseur doit se rendre (de -2 à 30) :"));
3 if (floor == 0) {
4   alert('Vous vous trouvez déjà au rez-de-chaussée.');
```

IV.3 Les boucles

Il est très utile de disposer d'une instruction permettant d'effectuer de manière répétitive une série d'opérations. JavaScript propose plusieurs types de boucles itératives. Pour chacune des boucles, la somme des nombres de 1 à 5 sera calculée.

IV.3.1 while

Formulation générale :

```
while ( <condition> ) {
    <action>;
    ...
}
```

Signification : Exécute-les <actions> aussi longtemps que la <condition> est vraie. Si la condition est fausse au début, aucune instruction n'est exécutée.

Exemple :

```
2  chiffre = 0 ;
3  somme = 0 ;
4  while ( chiffre < 5 ) {
5      chiffre = chiffre + 1;
6      somme = somme + chiffre;
7  }
8  |
```

IV.3.3 do ... while

Formulation générale :

```
do {
    <action>
    ...
} while ( <condition> )
```

Signification : Exécute-les <actions> aussi longtemps que la <condition> est vraie. Si la condition est fausse au début, les instructions seront quand même exécutées une seule fois.

Exemple:

```

2  chiffre = 0 ;
3  somme = 0 ;
4  do {
5  chiffre = chiffre + 1 ;
6  somme = somme + chiffre;
7  }
8  while ( chiffre < 5 )
9  |

```

IV.3.4 for

Formulation générale :

for (<expression de depart> ; <condition de continuation> ; <incrementation>) {
<action>

...

}

Exemple:

```

2  somme = 0 ;
3  for ( chiffre = 1 ; chiffre <= 5 ; chiffre = chiffre + 1 ) {
4  somme = somme + chiffre ;
5  }

```

```

1  <script>
2  for (var i = 0, prenoms = '', prenom; true; i++) {
3  prenom = prompt('Entrez un prénom :');
4  if (prenom) { prenoms += prenom + ' '; }
5  else { break; } }
6  alert('Il y a ' + i + ' prénoms :\n\n' + prenoms); // Les \n servent à faire des sauts de ligne
7  </script>

```

Signification : La boucle for se déroule de la manière suivante :

Tant que la condition de continuation est vraie/

- En partant de l'expression de départ (chiffre = 1) on exécute le contenu des accolades ;
- La variable (chiffre) est incrémentée (chiffre = chiffre + 1) et on exécute le contenu des accolades autant de fois que nécessaire ;
- La boucle ne prend fin que lorsque la condition de continuation devient fausse.

NB :

Il y a les fonctions ou variables natives (déjà existantes), mais on peut aussi en créer de nouvelles, selon la structure suivante :

```

1  <script>
2  var prenoms = '', prenom; // On crée une variable prenoms pour mémoriser
3  while (true) {
4  prenom = prompt('Entrez un prénom :'); // L'utilisateur entre chaque prenom
5  if (prenom) {
6  prenoms += prenom + ' '; // Ajoute le nouveau prénom ainsi qu'une espace
7  } else {
8  break; // On quitte la boucle
9  }
10 } alert(prenoms); // Affiche les prénoms à la suite
11 </script>

```

```

1  <script>
2  function byTwo() {
3  var result = parseInt(prompt('Donnez le nombre à multiplier par 2 :'));
4  alert(result * 2); }
5  byTwo(); // On appelle la fonction créée
6  alert('Vous en êtes à la moitié !'); // Puis un message intermédiaire
7  byTwo(); // Et appelle de nouveau la fonction
8  </script>

```

IV.3.5 La condition switch

Cette structure permet de gérer une courte liste de possibilités :

```

1  <script>
2  var drawer = parseInt(prompt('Choisissez le tiroir à ouvrir (1 à 4) :')); //on
3  précise bien le type de la valeur, ici un nombre avec la fonction parseInt()
4  switch (drawer) {
5  case 1: // on pose chaque cas l'un après l'autre ; on met des apostrophes si l'on vérifie des chaînes de caractères au lieu de nombres
6  alert('Contient divers outils pour dessiner : du papier, des crayons, etc.');
```

EXERCICE 3 : détermination de l'âge :

Fournir un commentaire selon l'âge de la personne. Vous devez fournir un commentaire sur 4 tranches d'âge qui sont les suivantes :

Tranche d'âge Exemple de commentaire

1 à 6 ans « Vous êtes un jeune enfant. »

7 à 11 ans « Vous êtes un enfant qui a atteint l'âge de raison. »

12 à 17 ans « Vous êtes un adolescent. »

18 à 120 ans « Vous êtes un adulte. »

Solution :


```

1  <script>
2  var age = parseInt(prompt('Quel est votre âge ?'));
3  if (1 <= age && age <= 6) {
4  alert('Vous êtes un jeune enfant. ');
5  } else if (7 <= age && age <= 11) {
6  alert ('Vous êtes un enfant qui a atteint l\'âge de raison. ');
7  } else if (12 <= age && age <= 17) {
8  alert ('Vous êtes un adolescent. ');
9  } else if (18 <= age && age <= 120) {
10 alert ('Vous êtes un adulte. ');
11 } else {
12 alert ('Erreur !!');
13 }
14 </script>

```

V. LES FONCTION ET PROCEDURES

On appelle fonction un sous-programme qui permet d'effectuer un ensemble d'instruction par simple appel de la fonction dans le corps du programme principal. Cette notion de sous-programme est généralement appelée fonction dans les langages autres que le Javascript (toutefois leur syntaxe est généralement différente...). Les fonctions et les procédures permettent d'exécuter dans plusieurs parties du programme une série d'instructions, cela permet une simplicité du code et donc une taille de programme minimale.

Dans JavaScript, les fonctions et les procédures sont définies par le mot clé `function`. La différence entre une fonction et une procédure est que la fonction retourne une valeur (numérique, booléen etc.), ce qui n'est pas le cas pour une procédure. Ce retour de valeur se fait par le mot clé `return`.

Avant d'être utilisée, une fonction doit être définie car pour l'appeler dans le corps du programme il faut que le navigateur la connaisse, c'est-à-dire qu'il connaisse son nom, ses arguments et les instructions qu'elle contient. La définition d'une fonction s'appelle "déclaration". La déclaration d'une fonction se fait grâce au mot clé `function` selon la syntaxe suivante:

```

function nom(parametre1,parametre2,...,parametreN) {
....
}

```

Exemple :

```
function carre(nombre) {  
    resultat = nombre * nombre ;  
    return(resultat)
```

Pour exécuter une fonction, il suffit de faire appel à elle en écrivant son nom (en respectant la casse) suivie d'une parenthèse ouverte (éventuellement des arguments) puis d'une parenthèse fermée.

Exemple :

```
Nom_De_La_Fonction();
```

V.1 Fonction *parseInt ()*

Du point de vue de JavaScript, tous les nombres sont des valeurs en virgule flottante. Cependant, vous pouvez faire appel à la fonction `parseInt ()` pour demander à JavaScript de ne prendre en compte que la partie entière de la valeur numérique (ce qui se trouve avant le point décimal), et donc d'éliminer le reste du nombre. Par exemple :

```
parseInt (100.33) ; // retourne 100
```

V.2 Fonction *parseFloat ()*

La fonction `parseFloat ()` demande à JavaScript de traiter un nombre comme étant décimal. Vous pouvez même vous en servir pour convertir une chaîne en nombre. Par exemple :

```
parseFloat("10"); // retourne 10
```

```
parseFloat(100.00); //retourne 100.00
```

```
parseFloat("10"); //retourne 10
```

VI. METHODES

Une méthode est une fonction associée à un objet, c'est-à-dire une action que l'on peut faire exécuter à un objet. Les méthodes des objets du navigateur sont des fonctions définies à l'avance par les normes HTML, on ne peut donc pas les modifier, il est toutefois possible de créer une méthode personnelle pour un objet que l'on a créé soi-même.

Prenons par exemple une page HTML, elle est composée d'un objet appelé document. L'objet document a par exemple la méthode **write()** qui lui est associée et qui permet de modifier le contenu de la page HTML en affichant du texte. Une méthode s'appelle un peu comme une propriété, c'est-à-dire de la manière suivante : **window.objet1.objet2.methode()**

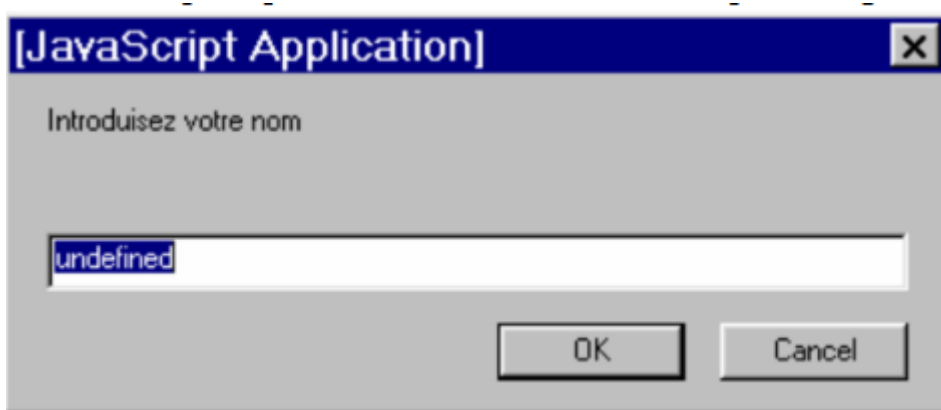
Dans le cas de la méthode **write()**, l'appel se fait comme suit : **window.document.write()**

VI.1 Affichage et introduction des données : alert et prompt

La fonction alert sert à afficher une valeur. Voici un exemple: *alert ("Hello World !");*



La fonction prompt sert à lire une valeur. Exemple : *x = prompt ("Introduisez votre nom");*



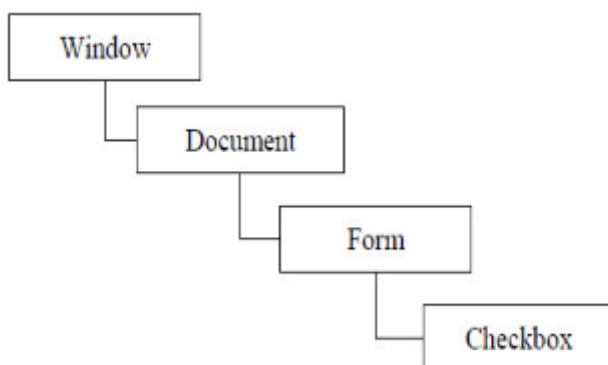
Après avoir cliqué sur OK, la variable x contient la chaîne de caractères qui a été introduite.

VII. LES OBJETS

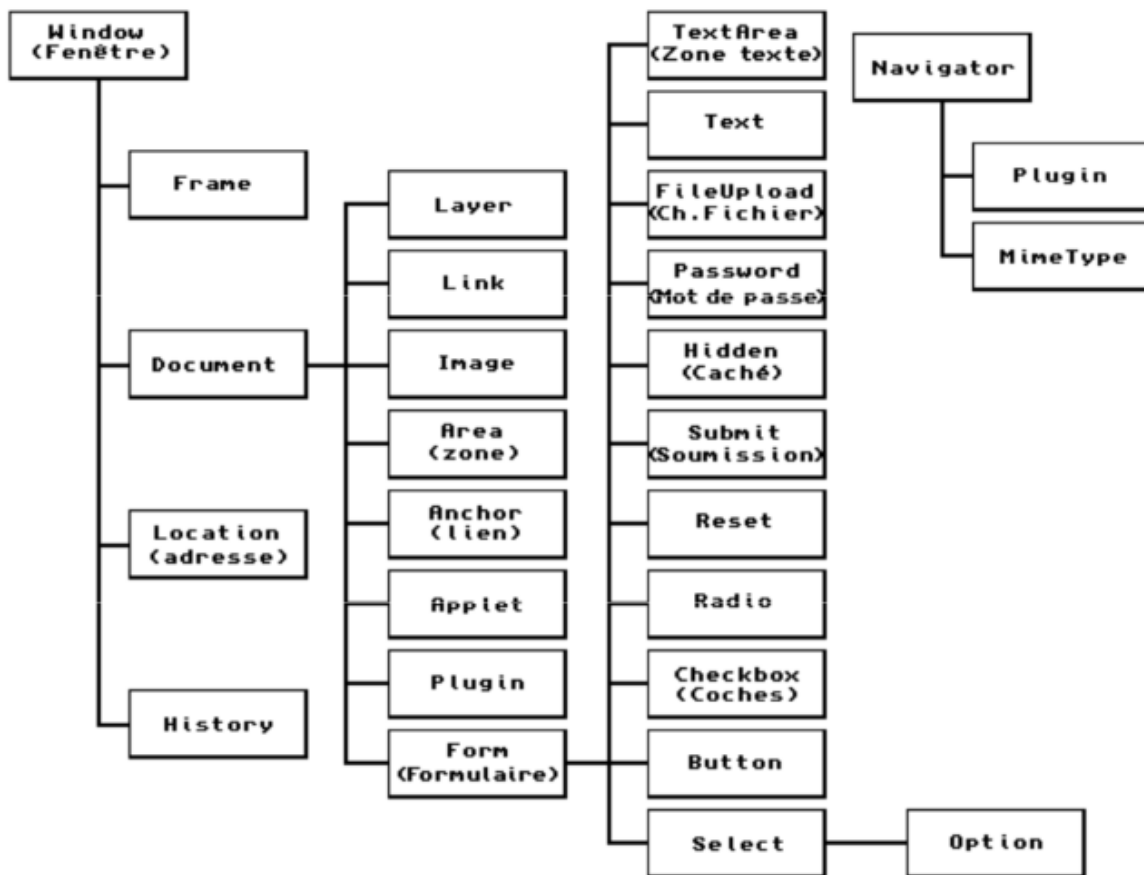
Javascript traite les éléments qui s'affichent dans votre navigateur comme des objets, c'est-à-dire des éléments classés selon :

- Une hiérarchie pour pouvoir les désigner précisément
- Auxquels on associe des propriétés

Par exemple, pour atteindre un bouton à l'intérieur d'un formulaire, la hiérarchie est :



La hiérarchie des objets est la suivante :



L'accès aux objets se fait par une notation par points. D'autre part, comme il peut y avoir plusieurs formulaires et plusieurs boutons dans chaque formulaire, certains objets sont automatiquement numérotés.

Ainsi, accéder au premier bouton du premier formulaire d'une page web se fait par la notation suivante : ***window.document.forms[0].checkbox[0]***

Le [0] représente ici le premier élément (le [1], le deuxième, et ainsi de suite). Il est aussi possible d'utiliser le nom de l'objet directement. Ainsi, si le formulaire s'appelle enquete (évitez les accents dans les noms d'objets) et le bouton s'appelle courrier, il sera possible d'y accéder de la manière suivante : ***window.document.enquete.courrier***

Enfin, il est aussi possible d'utiliser la notation :

window.document.form["enquête"].buton["courrier"]

VIII. LES EVENEMENTS

Les événements sont des actions de l'utilisateur qui vont pouvoir donner lieu à une interactivité. L'événement par excellence est le clic de souris, car c'est le seul que le HTML

gère. Grâce au Javascript il est possible d'associer des fonctions, des méthodes à des événements tels que le passage de la souris au-dessus d'une zone, le changement d'une valeur dans un champ, etc. Ce sont les gestionnaires d'événements qui permettent d'associer une action à un événement.

La syntaxe d'un gestionnaire d'événement est la suivante :

onEvenement="Action_Javascript_ou_Fonction()";

Les gestionnaires d'événements sont associés à des objets, et leur code s'insère dans la balise de ceux-ci. Chaque événement ne peut pas être associé à n'importe quel objet. Ainsi par exemple, l'événement ***OnChange*** ne peut pas s'appliquer à un lien hypertexte.

Le tableau ci-dessous présente les événements par liste alphabétique :

<i>Evénement</i>	Objets affectés	Description
<i>onAbort</i>	Images	S'exécute en cas d'arrêt de chargement, par appui sur le boutons stop ou par un clic prématuré sur un lien
<i>onBlur</i>	Fenêtres et tous les éléments de formulaire	S'exécute quand on quitte la fenêtre ou un objet de formulaire
<i>onChange</i>	Champs texte, zones texte, listes de sélection	S'exécute quand un élément de formulaire est modifié
<i>onClick</i>	Boutons, boutons radio, boutons submit et reset, liens	S'exécute quand on clique dans ou sur un élément
<i>onDragDrop</i>	Fenêtres	S'exécute quand on pose un élément à l'intérieur de la fenêtre du navigateur à l'aide la souris
<i>onError</i>	Images, fenêtres	S'exécute quand le chargement de l'image ou de la fenêtre provoque une erreur
<i>onFocus</i>	Fenêtres et tous les éléments de formulaire	S'exécute quand on sélectionne la fenêtre ou l'objet formulaire
<i>onKeyDown</i>	Documents, images, liens, zones texte	S'exécute quand une touche du clavier est pressée
<i>onKeyPress</i>	Documents, images, liens, zones texte	S'exécute quand on appuie et maintient une touche du clavier
<i>onKeyUp</i>	Documents, images, liens, zones texte	S'exécute quand on relâche une touche du clavier
<i>onLoad</i>	Documents	S'exécute quand le document se charge
<i>onMouseDown</i>	Documents, boutons, liens	S'exécute quand on clique avec le bouton de la souris
<i>onMouseMove</i>	rien par défaut	S'exécute quand on bouge la souris
<i>onMouseOut</i>	Cartes, liens	S'exécute quand le pointeur de la souris sort d'une zone de sélection graphique ou un lien

<i>onMouseOver</i>	liens	S'exécute quand le pointeur de la souris passe sur un lien
<i>onMouseUp</i>	Documents, boutons, liens	S'exécute quand on relâche le bouton de la souris
<i>onMove</i>	Fenêtres	S'exécute quand l'utilisateur ou un script bouge une fenêtre
<i>onReset</i>	Formulaires	S'exécute quand on "resete" un formulaire
<i>onResize</i>	Fenêtres	S'exécute quand l'utilisateur ou un script change la taille d'une fenêtre
<i>onSelect</i>	Champs ou zones texte	S'exécute quand on sélectionne une zone ou un champ texte (clavier ou souris)
<i>onSubmit</i>	Formulaire	S'exécute au moment de l'envoi d'un formulaire
<i>onUnLoad</i>	Documents	S'exécute quand on quitte le document

Le tableau ci-dessous résume les objets et les événements associés :

Objet	Evénements associables
Lien hypertexte	<i>onClick, onMouseOver, onMouseOut</i>
Page du navigateur	<i>onLoad, onUnload</i>
Bouton, Case à cocher, Bouton radio, Bouton Reset	<i>onClick</i>
Liste de sélection d'un formulaire	<i>onBlur, onChange, onFocus</i>
Bouton Submit	<i>onSubmit</i>
Champ de texte et zone de texte	<i>onBlur, onChange, onFocus, onSelect</i>

EXEMPLE 1:

```

1 <html>
2 <head>
3 <title>Ouverture d'une boîte de dialogue lors d'un click</title>
4 </head>
5 <body>
6 <a href="javascript:;" onClick="window.alert('Message d\'alerte a utiliser avec |moderation');">Cliquez ici!</a>
7 </body>
8 </html>

```

EXEMPLE 2 :

```

1 <script>
2 var myString = 'Ceci est une chaîne de caractères'; // On crée un objet String
3 alert(myString.length); // On affiche le nombre de caractères, au moyen de la
4 propriété « length »
5 alert(myString.toUpperCase()); // On récupère la chaîne en majuscules, avec la |méthode toUpperCase(), l'inverse étant la méthode
  toLowerCase()
6 </script>

```

IX. LES TABLEAUX

Après **Number**, **String** et **Boolean**, **Array** est un 4e objet natif de Javascript.

Un tableau, ou plutôt un array en anglais, est une variable qui contient plusieurs valeurs, appelées items. Chaque item est accessible au moyen d'un indice (index en anglais) et dont la numérotation commence à partir de 0.

```
1  <script>
2  var myArray = ['Rafael', 'Mathilde', 'Ahmed', 'Jérôme', 'Guillaume'];
3  // Le contenu se définit entre crochets, avec une virgule entre chaque valeur.
4  // La chaîne 'Rafael' correspond à l'indice 0, 'Mathilde' à l'indice 1...
5  alert(myArray[1]); // Affiche : « Laurence »
6  </script>
```

On peut modifier une valeur :

```
1  <script>
2  var myArray = ['Rafael', 'Mathilde', 'Ahmed', 'Jérôme', 'Guillaume'];
3  myArray[1] = 'Paul';
4  alert(myArray[1]); // Affiche : « Paul »
5  </script>
```

IX.1 Opérations sur les tableaux

On peut ajouter des items avec la méthode **push()** :

```
1  <script>
2  var myArray = ['Rafael', 'Mathilde'];
3  myArray.push('Ahmed'); // Ajoute « Ahmed » à la fin du tableau
4  myArray.push('Jérôme', 'Guillaume'); // Ajoute « Jérôme » et « Guillaume » à la fin du tableau
5  </script>
```

La méthode **unshift()** fonctionne comme **push()**, excepté que les items sont ajoutés au début du tableau. Les méthodes **shift()** et **pop()** retirent respectivement le premier et le dernier élément du tableau :

```
1  <script>
2  var myArray = ['Rafael', 'Mathilde', 'Ahmed', 'Jérôme', 'Guillaume'];
3  myArray.shift(); // Retire « Rafael »
4  myArray.pop(); // Retire « Guillaume »
5  alert(myArray); // Affiche « Mathilde,Ahmed,Jérôme »
6  </script>
```

On peut découper une chaîne de caractères en tableau avec **split()** :

```
1  <script>
2  var cousinsString = 'Jérôme Guillaume Paul',
3  cousinsArray = cousinsString.split(' '); // Avec les espaces, on a trois items
4  alert(cousinsString);
5  alert(cousinsArray);
6  </script>
```

On fait l'inverse avec **join()** :

```
1  <script>
2  var cousinsString_2 = cousinsArray.join('-');
3  alert(cousinsString_2); </script>
```

IX.2 Parcourir un tableau

On peut parcourir un tableau avec for :

```
1  <script>
2  var myArray = ['Rafael', 'Mathilde', 'Ahmed', 'Jérôme', 'Guillaume']; // La
3  length est de 5
4  for (var i = 0, c = myArray.length; i < c; i++) { // On crée la variable c pour que la condition ne soit pas trop lourde en caractères
5  alert(myArray[i]); // On affiche chaque item, l'un après l'autre, jusqu'à la
6  longueur totale du tableau
7  }
8  </script>
```

EXERCICE :

Demandez les prénoms aux utilisateurs et stockez-les dans un tableau. Pensez à la méthode push(). À la fin, il faudra afficher le contenu du tableau, avec alert(), seulement si le tableau contient des prénoms ; en effet, ça ne sert à rien de l'afficher s'il ne contient rien. Pour l'affichage, séparez chaque prénom par un espace. Si le tableau ne contient rien, faites-le savoir à l'utilisateur, toujours avec alert().

Solution :


```
1  <script>
2  .....
3  var prenom = [],
4  prenom;
5  while (prenom = prompt('Entrez un prénom :')) {
6  prenom.push(prenom); // Ajoute le nouveau prénom ainsi qu'un espace
7  }
8  if (prenom.length > 0) {
9  alert(prenom.join(' '));
10 } else {
11 alert('Il n\'y a aucun prénom en mémoire');
12 }
13 </script>
```